

DROO: 무선 전력 MEC의 온라인 오프로딩

Deep Reinforcement Learning for Online Computation Offloading in Wireless Powered Mobile-Edge Computing Networks

Liang Huang, Suzhi Bi, Ying-Jun Angela Zhang (2020)

신경망이 이진 후보를 제안하고 해석적 최적화가 물리 제약을 만족시키는 구조

무선 전력 MEC 시스템

하나의 AP가 에너지를 보내고 여러 WD의 오프로딩 데이터를 받는다

구성 요소	논문에서 말하는 역할	프레임에서 관측되는 값
AP	RF 에너지 전송과 MEC 계산	안정적인 전원과 높은 계산 능력
WD	에너지 수확 뒤 로컬 계산 또는 전송	AP와의 현재 채널 gain
무선 채널	수확 에너지와 uplink rate에 함께 영향	프레임 안에서는 고정, 프레임마다 변화

현재 채널 gain은 에너지 수확과 데이터 전송 성능을 동시에 바꾼다

한 프레임의 WPT와 오프로딩 시간

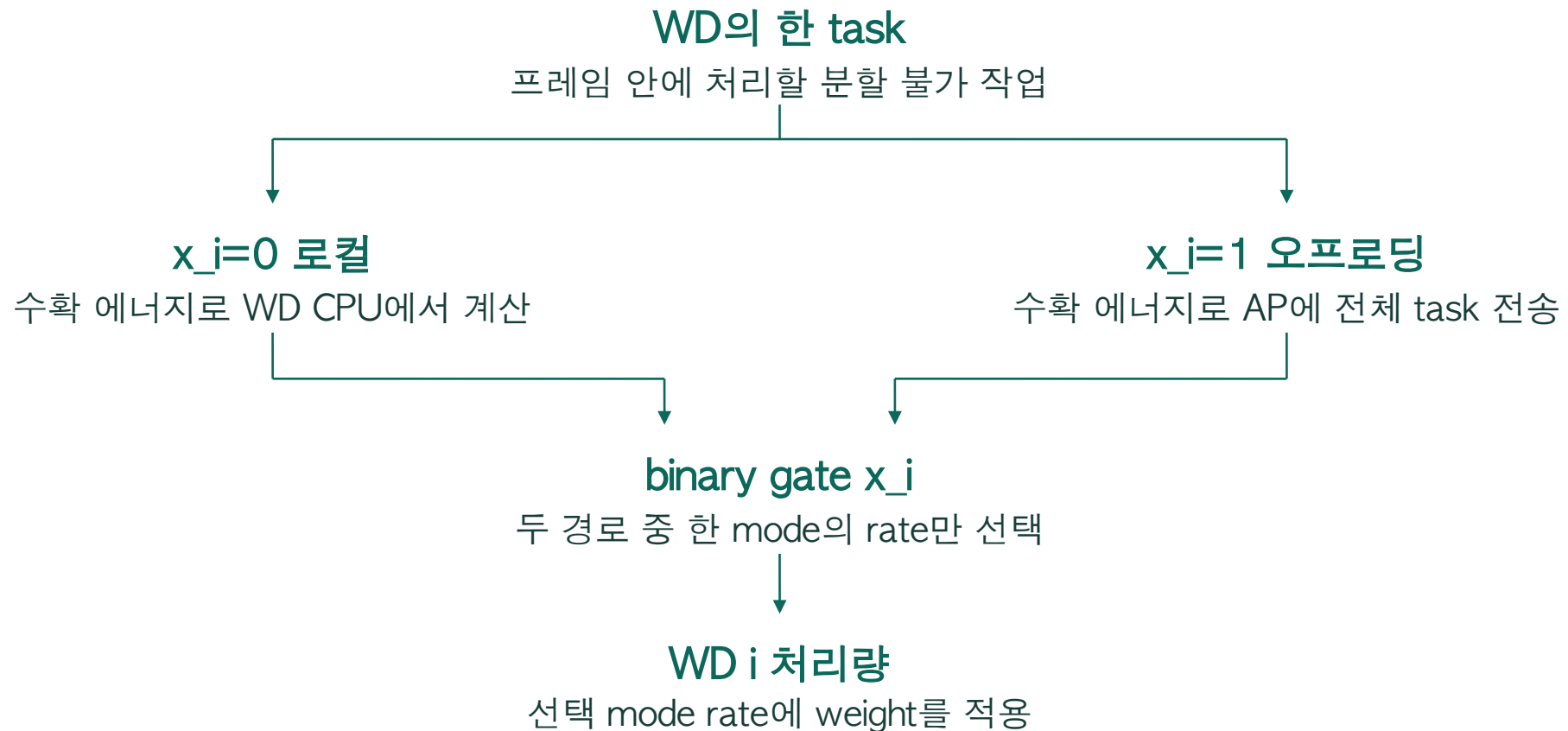
AP가 먼저 α 비율 동안 에너지를 보내고 선택된 WD가 남은 시간을 TDMA로 나눈다



TDD 모델에서는 에너지 전송이 먼저이고 오프로딩 전송은 그 뒤에 이어진다

각 task의 두 가지 실행 모드

논문은 task 분할 없이 로컬 계산과 완전 오프로딩 중 하나만 선택한다



모델이 단순화한 범위

가정

AP 계산 시간

결과 download

channel reciprocity

task 분할

계산에서의 처리

무시

무시

같은 gain 표기

허용하지 않음

해석할 때의 경계

AP가 WD보다 훨씬 빠르다는 전제

반환 데이터가 짧다는 전제

uplink와 downlink가 같다는 단순화

로컬 또는 전체 오프로딩만 비교

DROO의 성능 주장은 이 시스템 모델과 시간 가정 안에서 읽어야 한다

지수적 탐색 공간의 원인

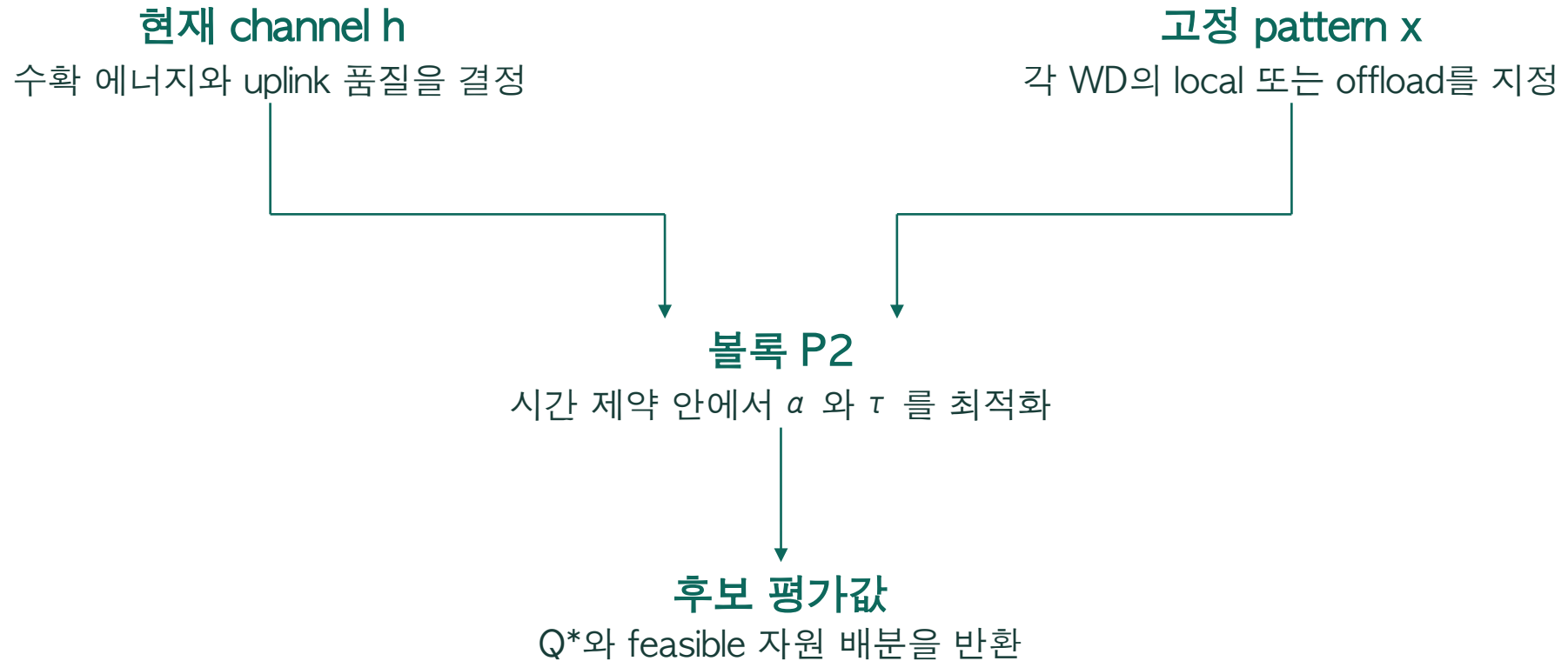
N개 WD의 이진 선택은 2^N 개의 local-offload pattern을 만든다

- WD가 3개면 가능한 이진 pattern은 8개다.
- WD가 20개면 가능한 pattern은 1,048,576개다.
- WD가 30개면 가능한 pattern은 1,073,741,824개다.
- 연속 변수 α 와 τ 보다 먼저, 이진 vector x 를 찾는 단계가 조합 폭발을 만든다.
- 채널이 달라질 때마다 coherence time 안에서 같은 문제를 다시 풀어야 한다.

DROO는 2^N pattern 전체를 평가하지 않고 DNN 주변의 K개 후보만 평가한다

고정된 이진 pattern과 P2

x 를 고정하면 남은 α 와 τ 최적화는 블록 문제로 바뀐다



P2는 무엇을 최대화하는가

고정된 x 에서 $Q = \sum w_i[(1-x_i)r_{L,i}^* + x_i r_{O,i}^*]$ 를 최대화한다

구성	수학적 조건	의미
선택 변수	α, τ_i	WPT와 장치별 전송 시간 비율
시간 제약	$\alpha + \sum \tau_i \leq 1; \alpha, \tau_i \geq 0$	프레임 시간을 함께 사용
에너지 처리	각 모드의 $r_{L,i}^*, r_{O,i}^*$ 에 반영	수확 에너지 전량 사용에서 유도
볼록 최적화	고정 x 의 오목 목적함수 최대화	선형 시간 제약 아래 후보별 해 계산

이진 후보는 DNN-OPQ가 만들고 연속 시간 배분은 P2가 최적화한다

DROO에서 구분해야 할 네 값

값	형식	역할	아직 아닌 것
현재 h_t	연속 channel vector	DNN 입력	미래 channel 예측
relaxed $x_{\hat{t}}$	0과 1 사이 score	offload 순위 제안	실행 가능한 최종 action
binary x^k	0 또는 1 vector	P2에 넣을 후보	자원 배분이 끝난 해
a^*, τ^*	연속 시간 배분	P2가 계산	DNN의 직접 출력

DNN score, binary action, 자원 배분을 한 단계로 합쳐 이해하면 안 된다

OPQ의 네 후보 생성 예시

논문의 relaxed output $\{0.2, 0.4, 0.7, 0.9\}$ 에서 $K=4$ 후보를 만든다

후보	적용한 경계	binary action	변경되는 위치
x^1	score > 0.5이면 1	$\{0, 0, 1, 1\}$	0.5와 같으면 0
x^2	threshold = 0.4	$\{0, 1, 1, 1\}$	경계값 0.4는 1
x^3	threshold = 0.7	$\{0, 0, 0, 1\}$	경계값 0.7은 0
x^4	threshold = 0.2	$\{1, 1, 1, 1\}$	경계값 0.2는 1

OPQ는 0.5와 가까운 score의 순서를 따라 서로 다른 binary 후보를 만든다

OPQ에서 경계와 같은 값은 어떻게 처리하는가

기본 후보와 나머지 후보의 동률 처리 규칙을 구분한다

경우	이진 값	예시
기본 후보	$\text{score} > 0.5 \rightarrow 1$; 그 외 0	$\text{score} = 0.5 \rightarrow 0$
나머지 후보: $\text{score} > b$	1	$b = 0.4, \text{score} = 0.7 \rightarrow 1$
나머지 후보: $\text{score} < b$	0	$b = 0.7, \text{score} = 0.4 \rightarrow 0$
$\text{score} = b, b \leq 0.5$	1	$b = \text{score} = 0.4 \rightarrow 1$
$\text{score} = b, b > 0.5$	0	$b = \text{score} = 0.7 \rightarrow 0$

일반 $K \leq 2^N$ 과 이 OPQ의 $K \leq N+1$ 은 서로 다른 범위다

order-preserving의 정확한 뜻

- $\text{score}_i \geq \text{score}_j$ 이면, 모든 후보에서 $x_i^k \geq x_j^k$ 다.
- score 순위를 유지하면서 threshold를 바꿔 K개의 후보를 만든다.
- KNN 후보보다 서로 떨어진 pattern을 만들어 탐색 다양성을 높이려는 설계다.
- 모든 2^N action을 확인하지 않으므로 전역 최적 action 포함을 보장하지 않는다.

OPQ는 DNN의 상대 순위를 보존하는 후보 생성 규칙이지 exhaustive search가 아니다

한 프레임의 online 실행 경로

- | | | |
|----|---------------|---|
| 01 | channel 관측 | 프레임 시작에 현재 h_t 를 읽는다. |
| ↓ | | |
| 02 | relaxed score | DNN이 $\hat{x}_t$ 를 출력한다. |
| ↓ | | |
| 03 | K개 후보 | OPQ가 binary x_t^k 를 만든다. |
| ↓ | | |
| 04 | P2 평가 | 각 후보의 Q^* 와 α^* , τ^* 를 계산한다. |
| ↓ | | |
| 05 | 선택과 실행 | 가장 큰 Q^* 의 x_t^* 와 자원을 실행한다. |

실행 action은 DNN 출력이 아니라 K번의 P2 평가 뒤에 결정된다

P2는 후보별 자원 최적화·평가를 맡는다

‘analytical critic’은 이 역할을 설명하는 교육용 이름이며 원문의 P2를 가리킨다

구성	입력	출력	보장하거나 학습하는 것
DNN	현재 h_t	relaxed $x_{\hat{t}}$	binary 후보의 순위를 학습
OPQ	relaxed score	K개 binary pattern	순서를 보존한 후보 생성
P2	h_t 와 x_t^k	Q^*, a^*, τ^*	시간 제약을 만족하는 최적 배분
argmax	K개의 Q^*	x_t^*	후보 집합 안에서 최고값 선택

P2는 learned value network가 아니라 후보마다 호출되는 볼록 optimizer다

실행 경험에서 다음 policy까지

online 선택 결과를 label로 저장하고 매 d 프레임마다 replay batch로 DNN을 갱신한다



runtime은 매 프레임 수행되지만 DNN training은 d 프레임 간격으로 수행된다

adaptive K의 갱신 규칙

$t \bmod D = 0$ 일 때, 현재 프레임 이전 D개의 선택 순위로 갱신한다



adaptive K는 P2 호출 수를 줄이는 경험적 cost-quality 조절이며 최적성 보장이 아니다

교육용 계산: 수확 에너지를 모두 쓰는 고정 후보

$T=1$ s, $x=[1,0]$, $\alpha=0.50$, $\tau_1=0.25$ 인 모형 일치 지점을 계산한다

항목	가정 또는 계산	해석
시간	$\alpha + \tau_1 = 0.75 \leq 1$	feasible일 뿐 최적해 주장은 아님
수확 조건	$\mu_{Ph1}=0.20$ W, $\mu_{Ph2}=0.12$ W	$E1=0.10$ J, $E2=0.06$ J
WD1: offload	$P1^*=E1/(\tau_1 T)=0.40$ W	0.25 s 동안 0.10 J 전력 사용
WD2: local	$f2^*=(0.06/(\kappa_2 T))^{(1/3)}$	T 전체에서 0.06 J 전력 사용

P2는 임의의 에너지 사용량을 고르지 않고 α 와 τ 를 바꿔 weighted rate를 최대화한다

수치 실험의 조건

범주	논문 설정	해석에 필요한 조건
무선 환경	Rayleigh fading, 915 MHz, 2 MHz	프레임별 독립 channel
전력	AP 3 W, 수확 효율 0.51	Powercast 장치 parameter
DNN	hidden 120과 80	TensorFlow 1.0, Adam 0.01
replay	memory 1024, batch 128	training interval d=10
실행 장비	i5-4570 3.2 GHz, RAM 12 GB	CPU latency 비교의 기준

결과 수치와 latency는 이 simulation, model, hardware 조건에 묶여 있다

OPQ와 KNN 결과의 조건부 비교

N=10 fading simulation의 200-frame 이동평균으로 수렴을 비교했다

조건	논문 관측	말할 수 있는 범위
N=10, K=N	두 방법 모두 normalized rate 1에 수렴	충분한 후보를 둔 simulation 결과
N=10, K=2	두 방법 모두 약 0.95에 수렴	작은 K에서는 suboptimal 가능
K가 2 이상	OPQ가 KNN보다 빠르게 수렴	논문의 fading simulation 안의 비교
K=1	두 방법 모두 수렴하지 않음	action selection이 없어 policy 개선이 막힘

OPQ의 장점은 논문 실험의 수렴 속도이며 보편적 우월성 증명은 아니다

computation rate 비교의 조건

평가 단계	조건	논문 결과
DROO 학습	24,000개 독립 channel	policy 수렴 뒤 평가
평가 평균	새 channel 6,000개	각 N에서 평균 rate 계산
기준 방법	CD, LR, all-local, all-edge	DROO는 CD와 유사한 rate
정규화 기준	N=10: 열거 최적; N=20,30: CD	median ≈ 1 ; 신뢰구간은 대부분 .99 이상

N=10은 열거 최적값, 큰 N은 CD benchmark와의 조건부 비교다

N=30에서 보고된 CPU latency

30,000개 독립 channel 평균이며 action generation과 DNN training을 포함한다

방법	평균 latency	2초 frame 대비	비고
DROO fixed K=N	0.31 s	15.5%	P2를 N번 평가
DROO adaptive K	0.059 s	약 3%	D=32
Coordinate Descent	3.8 s	190%	frame보다 김
Linear Relaxation	0.81 s	40.5%	iterative solver

0.059초는 보편적 real-time 보장이 아니라 논문 장비와 2초 frame 가정의 결과다

적용 범위와 남은 한계

- binary policy는 한 task의 partial offloading을 허용하지 않는다.
- AP 계산 시간과 결과 download 시간은 모델에서 제외한다.
- state는 channel gain이며 queue·battery backlog를 포함하지 않는다.
- 각 후보마다 P2를 풀어야 하므로 analytical optimization 비용은 남는다.
- WD mobility는 channel 분포를 바꿔 policy 수렴을 어렵게 할 수 있다.

DROO는 후보 탐색을 줄이지만 모델 가정과 P2 호출 비용까지 제거하지는 않는다

DROO와 LyDROO의 경계

비교 항목	DROO	LyDROO
문제 환경	무선 전력 MEC	stochastic task arrival MEC
state	현재 channel gain	channel·실제 큐·가상 전력 큐
목적	프레임별 weighted rate	장기 처리량과 queue 안정성
candidate	OPQ	OPQ + noisy OPQ (NOP)
이론 경계	queue 안정성 보장 없음	가정 아래 queue 안정성 분석

queue backlog와 Lyapunov 안정성은 후속 LyDROO의 문제이며 DROO에 소급되지 않는다

DROO의 핵심 계산 구조

01	현재 channel	h_t 가 수확과 uplink 조건을 정한다.
↓		
02	DNN proposal	relaxed score로 offload 순위를 제안한다.
↓		
03	OPQ candidates	순위를 보존한 K개 binary action을 만든다.
↓		
04	후보별 P2	각 후보의 feasible 자원과 Q^* 를 계산한다.
↓		
05	선택과 학습	최고 후보를 실행하고 pair를 replay에 저장한다.

학습이 이진 후보를 줄이고 P2가 각 후보 안의 연속 배분을 최적화한다

핵심 notation

기호	뜻	결정 주체
h_t	현재 프레임 channel gain vector	환경 관측
$x_{\hat{t}}$	DNN의 relaxed offload score	DNN
x_t^k	k번째 binary candidate	OPQ
x_t^*	선택된 binary action	K개 Q^* 의 argmax
a^*, τ^*	WPT와 offload 시간	P2

공개 출처

자료	공개 위치	이 자료에서의 용도
DROO journal paper	IEEE DOI 10.1109/TMC.2019.2928811	시스템, 알고리즘, 실험 근거
DROO manuscript	arXiv 1808.01977	저널판 이전 공개 원고 기록
DROO code	github.com/revenol/DROO	저자 공개 구현
LyDROO paper	IEEE DOI 10.1109/TWC.2021.3085319	후속 연구 경계 비교